CHƯƠNG 4:
TÍNH TOÁN BẤT ĐỒNG BỘ TRONG PYTHON

Phần I: Tổng quan các Mô hình Tính toán Nền tảng
(Tuần tự • Đồng thời • Song song • Bất đồng bộ)

Giảng viên hướng dẫn: ThS. Lê Anh Nhã Uyên (lanuyen@ctuet.edu.vn)
Khoa Công nghệ Thông tin — Trường Đại học Kỹ thuật - Công nghệ Cần Thơ

GIỚI THIỆU

Phân định Bản chất: Xử lý Đồng thời khác biệt với Xử lý Song song

• Luận điểm kinh điển (Rob Pike, 2012): "Đồng thời là giải quyết nhiều việc cùng lúc; Song song là thực hiện nhiều việc tại cùng một thời điểm."

• Đồng thời là về CẤU TRÚC: Phân rã bài toán thành các phân hệ độc lập có thể điều phối và tiến triển đan xen.

• Song song là về THỰC THI: Vận hành đồng thời nhiều phép tính trên các mạch phần cứng vật lý độc lập.

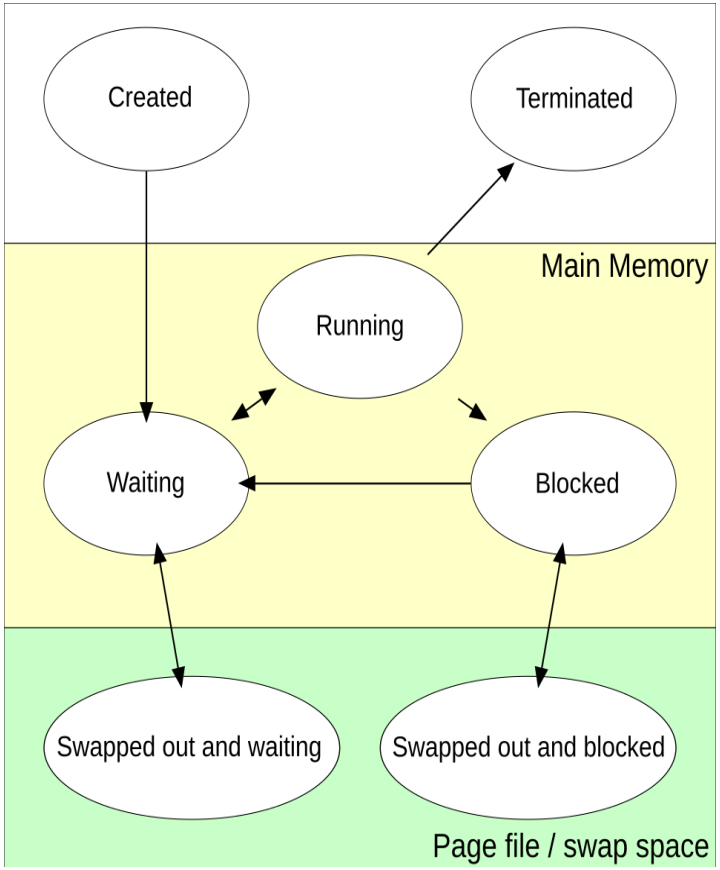
• Ý nghĩa kiến trúc: Một chương trình có thể đạt tính đồng thời cao trên hệ thống chỉ có một bộ vi xử lý đơn nhân.

Parallelism	CPU Core 1 ---------->	Task1
	CPU Core 2 ---------->	Task2
Concurrency	CPU Core —---- —----->	Task1
	—----- —-->	Task2

GIỚI THIỆU

Tiến trình: Đơn vị Cô lập Tài nguyên của Hệ điều hành

- **Định nghĩa chuẩn học thuật: Tiến trình là một thể hiện của chương trình đang trong trạng thái thực thi.**
- Không gian địa chỉ cô lập: Mỗi tiến trình sở hữu mã định danh riêng biệt và bảng trang ảo độc lập, ngăn ngừa can thiệp chéo.
- Cấu trúc bốn vùng nhớ: Vùng mã lệnh, vùng biến tĩnh, vùng nhớ cấp phát động và ngăn xếp cục bộ.
- Chi phí chuyển đổi ngữ cảnh: Hệ điều hành phải lưu khối điều khiển tiến trình, nạp lại thanh ghi và xóa sạch bộ đệm chuyển đổi địa chỉ.



GIỚI THIỆU

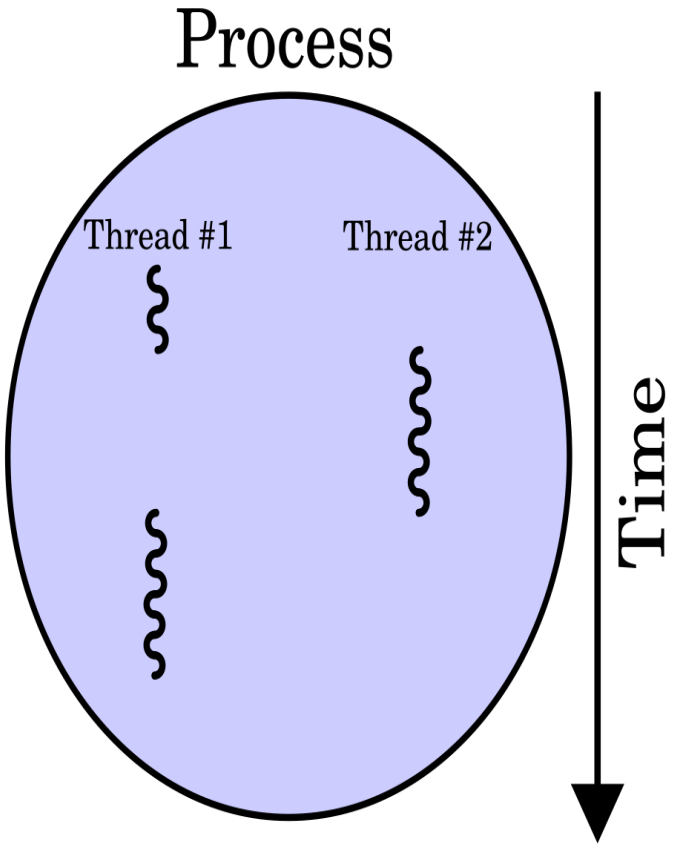
Luồng: Đơn vị Thực thi Nhẹ và Chia sẻ Không gian Bộ nhớ

• Định nghĩa chuẩn học thuật: Luồng là đơn vị nhỏ nhất được bộ điều phối của hệ điều hành phân bổ thời gian vi xử lý.

• Chia sẻ không gian nhớ: Các luồng trong cùng một tiến trình dùng chung vùng mã lệnh, vùng nhớ động và các tài nguyên mở.

• Khởi tạo và hoán đổi tối ưu: Chi phí chuyển ngữ cảnh luồng nhẹ hơn tiến trình từ 10 đến 20 lần do không phải đổi không gian trang nhớ.

• Giới hạn của trình thông dịch CPython: Cơ chế Khóa thông dịch toàn cục (GIL) chỉ cho phép một luồng duy nhất thực thi mã bytecode tại một thời điểm.



GIỚI THIỆU

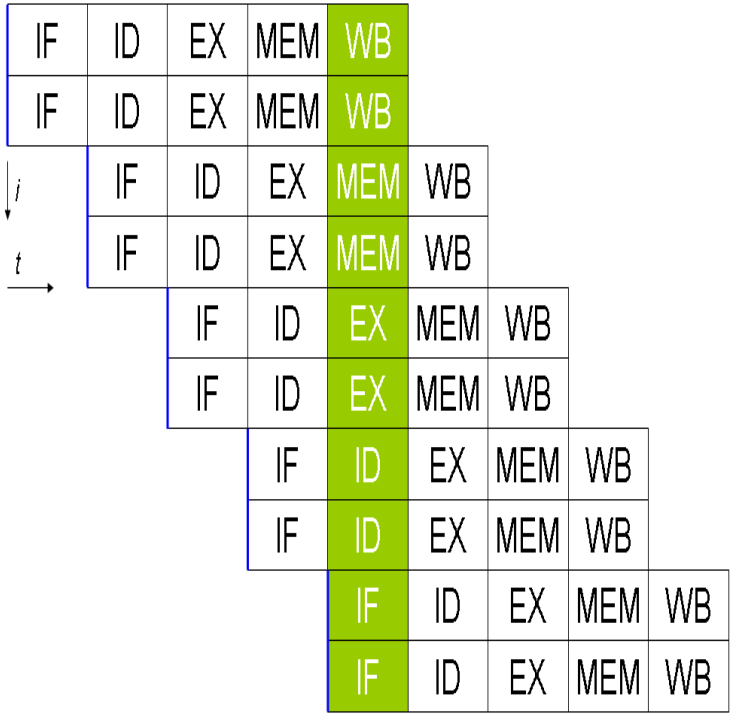
Khoảng cách Tốc độ Phần cứng và Sự Lãng phí của Vào/Ra Phong tỏa

• Nút thắt cổ chai Vào/Ra: Tốc độ xử lý của mạch vi xử lý vượt trội hơn hàng triệu lần so với các thiết bị lưu trữ và đường truyền mạng.

• Tương quan độ trễ: Bộ đệm cấp 1 (L1 Cache) phản hồi trong ~1 nano giây, trong khi một gói tin mạng cần đến ~50.000.000 nano giây.

• Thiệt hại do phong tỏa: Lời gọi hàm đồng bộ truyền thống khiến vi xử lý rơi vào trạng thái ngủ đông hàng chục triệu chu kỳ vô ích.

• Mục tiêu của tính toán bất đồng bộ: Thu hồi ngay quyền điều khiển của vi xử lý khi gặp tác vụ chờ dữ liệu để phục vụ yêu cầu tính toán khác.



GIỚI THIỆU

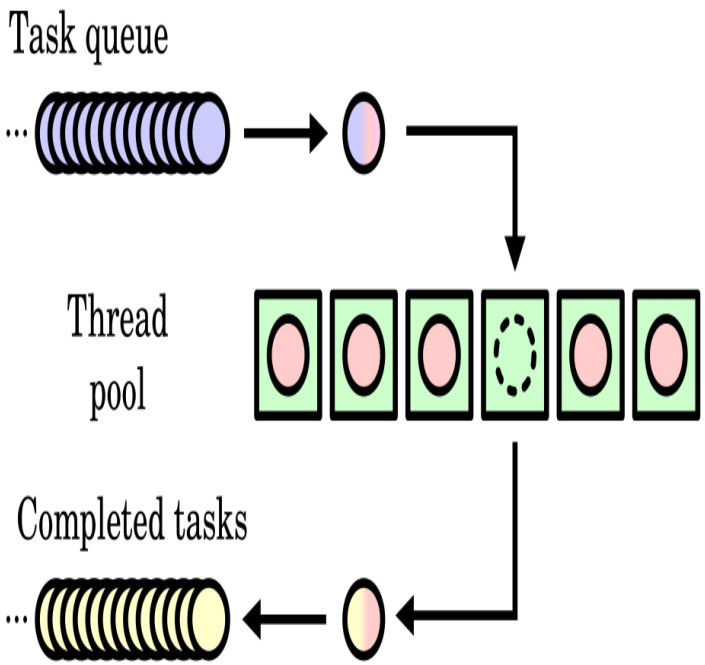
Bài toán C10K và Sự Ra đời của Kiến trúc Hướng Sự kiện

• Thách thức C10K (Dan Kegel, 1999): Làm thế nào để một máy chủ duy nhất có thể duy trì và phục vụ đồng thời 10.000 kết nối?

• Giới hạn của mô hình mỗi máy trạm một luồng: Cấp phát 10.000 luồng làm cạn kiệt bộ nhớ và gây quá tải nghiêm trọng cho bộ lập lịch của hệ điều hành.

• Đột phá kỹ thuật dồn kênh Vào/Ra: Hệ điều hành cung cấp cơ chế thông báo sự kiện tập trung (epoll trên Linux, kqueue trên BSD/macOS, IOCP trên Windows).

• Bước chuyển mình kiến trúc: Một luồng đơn duy nhất có thể quản lý hàng vạn kết nối đồng thời với lượng bộ nhớ tiêu thụ tối thiểu.



GIỚI THIỆU

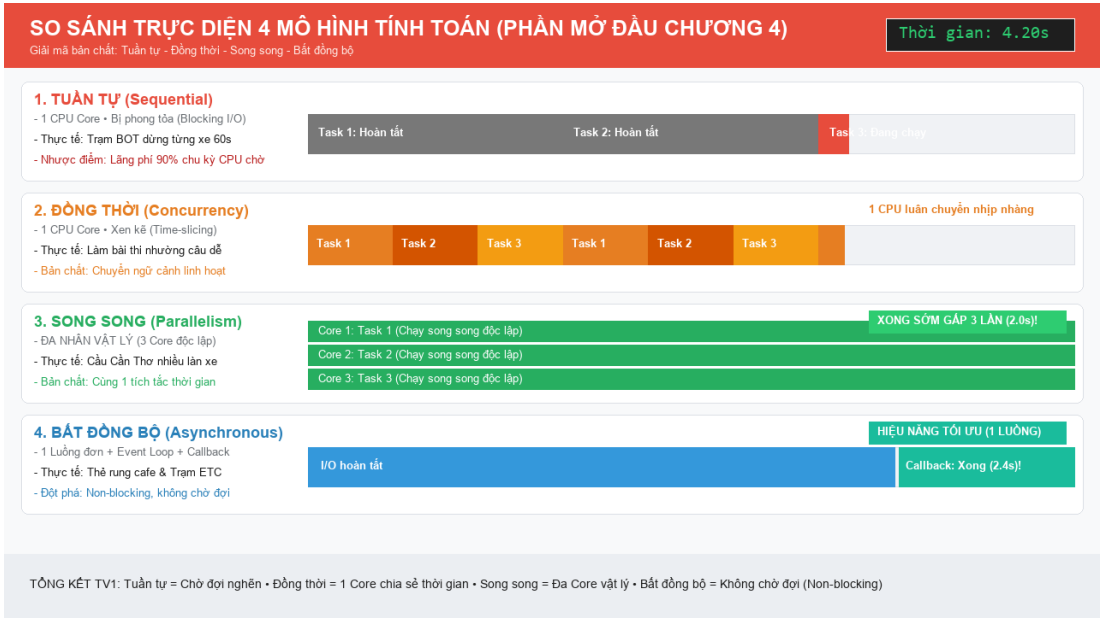
Không gian Kiến trúc: Phân loại Bốn Mô hình Tính toán Nền tảng

• **Phân loại hai chiều: Cấu trúc Phần cứng (Đơn nhân / Đa nhân) và Cơ chế Điều phối (Phong tỏa / Phi phong tỏa):**

Cấu trúc Phần cứng	Cơ chế Phong tỏa (Blocking)	Cơ chế Phi phong tỏa (Non-blocking)
Bộ vi xử lý Đơn nhân (Đơn lõi CPU)	XỬ LÝ TUẦN TỰ (Sequential) • Thực thi đơn tuyến tuần tự từng chỉ thị lệnh. • Vi xử lý bị đóng băng hoàn toàn khi chờ dữ liệu vào/ra.	BẤT ĐỒNG BỘ ĐƠN LUỒNG (Asyncio) • 1 Luồng kết hợp Vòng lặp sự kiện (Event Loop). • Giải phóng vi xử lý ngay khi chờ I/O để phục vụ việc khác.
Bộ vi xử lý Đa nhân (Đa lõi phần cứng)	XỬ LÝ SONG SONG & ĐA LUỒNG • Nhiều nhân phần cứng thực thi đồng thời thực sự. • Từng luồng vẫn có thể bị phong tỏa nếu chờ Vào/Ra.	HỆ THỐNG PHÂN TÁN / LAI GHÉP • Mỗi nhân vận hành một Vòng lặp sự kiện riêng biệt. • Tối đa hóa thông lượng phục vụ hàng vạn kết nối đồng thời.

GIỚI THIỆU

Mô phỏng Động học: Đối chiếu Thời gian Thực thi của Bốn Mô hình



Đặc tính Động học Từng Mô hình:

- Tuần tự: Thực thi nối tiếp từng tác vụ; vi xử lý bị phong tỏa hoàn toàn khi chờ Vào/Ra; tổng thời gian đạt đỉnh (6.0 giây).
- Đồng thời: Một nhân vi xử lý phân chia lát cắt thời gian; chuyển đổi ngữ cảnh linh hoạt giữa các tác vụ đang chờ.
- Song song: Ba nhân vật lý độc lập xử lý cùng lúc ba tác vụ; đạt tốc độ nhanh nhất về mặt tính toán thuần túy (2.0 giây).
- Bất đồng bộ: Một luồng đơn kết hợp vòng lặp sự kiện; giải phóng hoàn toàn vi xử lý trong thời gian chờ dữ liệu (2.4 giây).

- **Kết luận học thuật: Mô phỏng trực quan làm rõ ranh giới giữa năng lực tính toán phần cứng và giải thuật điều phối phần mềm — Cung cấp nền tảng tư duy vững chắc trước khi tiếp cận kỹ thuật lập trình.**

GIỚI THIỆU

Phân tích Động học: Chi phí Chuyển đổi Ngữ cảnh & Tác vụ Vào/Ra

1. Cơ chế Tác vụ Nghẽn Vào/Ra (I/O-Bound)

• Thời gian chờ áp đảo: Thời gian chờ đường truyền mạng hoặc ổ đĩa chiếm trên 95% tổng vòng đời tác vụ.

• Khử thời gian nhàn rỗi: Mô hình bất đồng bộ tách rời lời gọi yêu cầu khỏi việc chờ kết quả, triệt tiêu thời gian vi xử lý bị đóng băng.

• Cơ chế nhường quyền: Tác vụ chủ động nhường vi xử lý khi phát sinh thao tác chờ và chỉ được nạp lại khi có tín hiệu sẵn sàng.

2. Chi phí Chuyển đổi Ngữ cảnh Thực thi

• Ngữ cảnh nhân hệ điều hành: Chuyển đổi giữa các luồng hệ điều hành tiêu tốn hàng nghìn chu kỳ xung nhịp để lưu bảng trạng thái và nạp lại thanh ghi.

• Ngữ cảnh không gian người dùng: Chuyển đổi giữa các hàm đồng quy chỉ thay đổi con trỏ khung ngăn xếp, chi phí chỉ vài chục nano giây.

• Tối ưu hóa bộ nhớ: Triệt tiêu sự lãng phí cấp phát ngăn xếp tĩnh của luồng hệ điều hành, cho phép duy trì hàng vạn tác vụ đồng thời.

GIỚI THIỆU

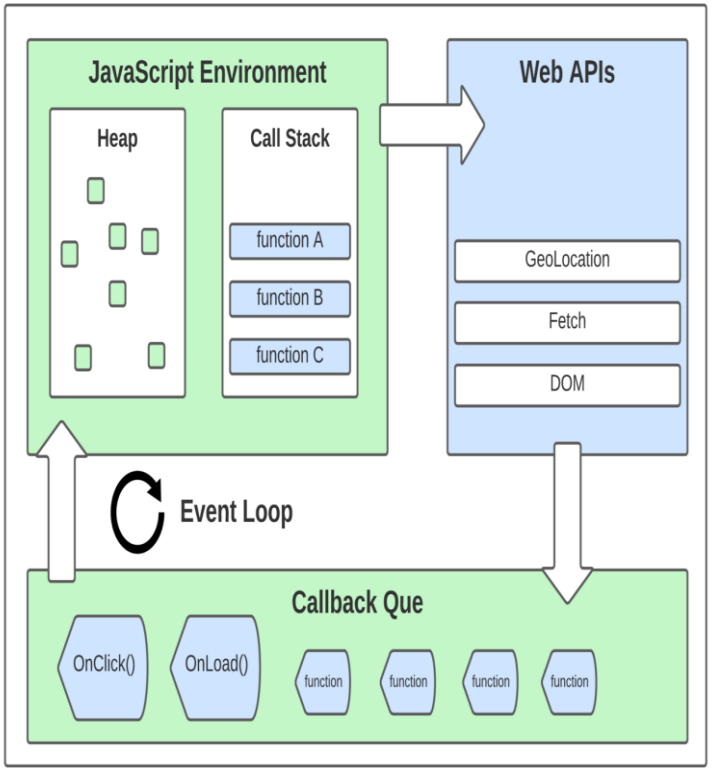
Mẫu Thiết kế Reactor: Kiến trúc Xương sống của Vòng lặp Sự kiện

• Định nghĩa chuẩn kiến trúc: Bộ phản ứng là mẫu thiết kế phục vụ việc tiếp nhận, phân tách và điều phối đồng thời các yêu cầu dịch vụ từ nhiều luồng dữ liệu vào.

• Bốn thành phần cốt lõi: Bộ định danh tài nguyên (Handles), Bộ phân kênh sự kiện đồng bộ, Bộ khởi tạo điều phối trung tâm và Các bộ xử lý sự kiện.

• Ưu điểm kiến trúc: Quản lý hàng vạn kết nối đồng thời trên một luồng duy nhất mà không tốn chi phí khóa đồng bộ.

• Cơ sở thực thi: Đây chính là nguyên lý vận hành nền tảng của vòng lặp sự kiện trong thư viện asyncio của Python.



GIỚI THIỆU

Đặc tả Chuẩn PEP 492: Hàm Đồng quy Tự nhiên và Cú pháp Async/Await

• Chuẩn hóa ngôn ngữ: Kể từ phiên bản 3.5, Python chính thức nâng cấp hàm đồng quy thành một khái niệm độc lập ở tầng cú pháp.

• Lập lịch hợp tác tự nguyện: Tác vụ chủ động tạm dừng tại vị trí khai báo await để nhường quyền kiểm soát vi xử lý cho vòng lặp.

• Cấu trúc siêu nhẹ: Hàng vạn hàm đồng quy có thể cùng chia sẻ một luồng duy nhất; mỗi hàm chỉ tiêu tốn khoảng vài kilobyte bộ nhớ.

• Bảo toàn trạng thái không gian người dùng: Khung ngăn xếp thực thi được lưu trữ trực tiếp trong bộ nhớ ứng dụng, cho phép phục hồi trạng thái tức thì.

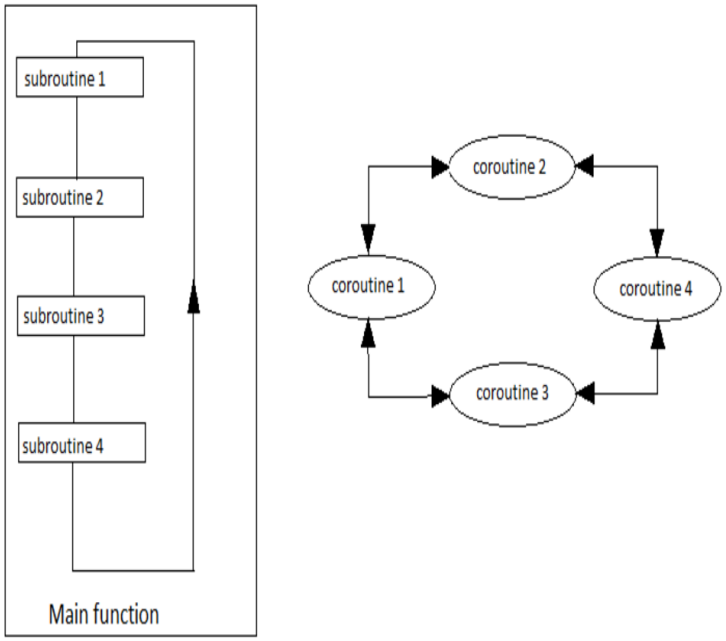


figure 1: diagram showing the difference between coroutines and subroutines. The main function controls the execution and order of subroutines, while coroutines are collaboravive and interdependant.

GIỚI THIỆU

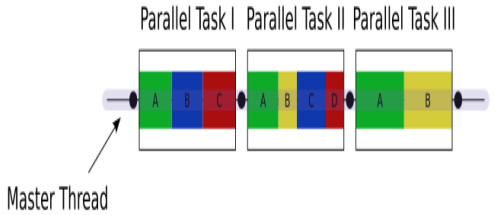
Hiện tượng Tranh chấp Dữ liệu và Các Hiểm họa Bộ nhớ Chia sẻ

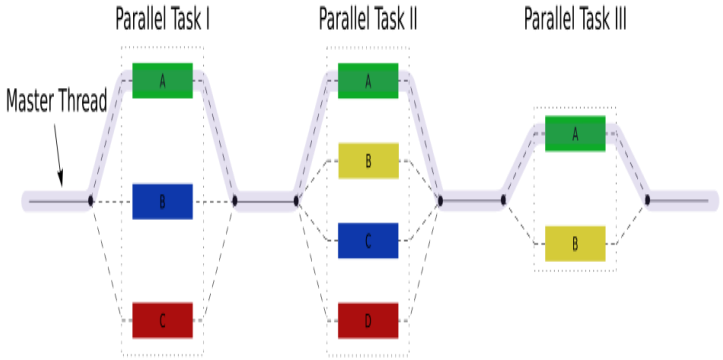
• Định nghĩa hiện tượng tranh chấp: Xảy ra khi kết quả của chương trình phụ thuộc vào thứ tự thực thi ngẫu nhiên giữa các luồng.

• Xung đột truy cập dữ liệu: Xuất hiện khi nhiều luồng cùng thao tác trên một ô nhớ dùng chung và có ít nhất một thao tác ghi mà không có đồng bộ.

• Lỗi bất định phụ thuộc thời gian: Các sai lệch logic chỉ xuất hiện ngẫu nhiên khi hệ thống chịu tải cao và biến mất khi kích hoạt chế độ gỡ lỗi.

• Nguyên lý an toàn bộ nhớ: Hạn chế chia sẻ trạng thái bộ nhớ biến đổi; ưu tiên truyền thông điệp thông qua hàng đợi an toàn.

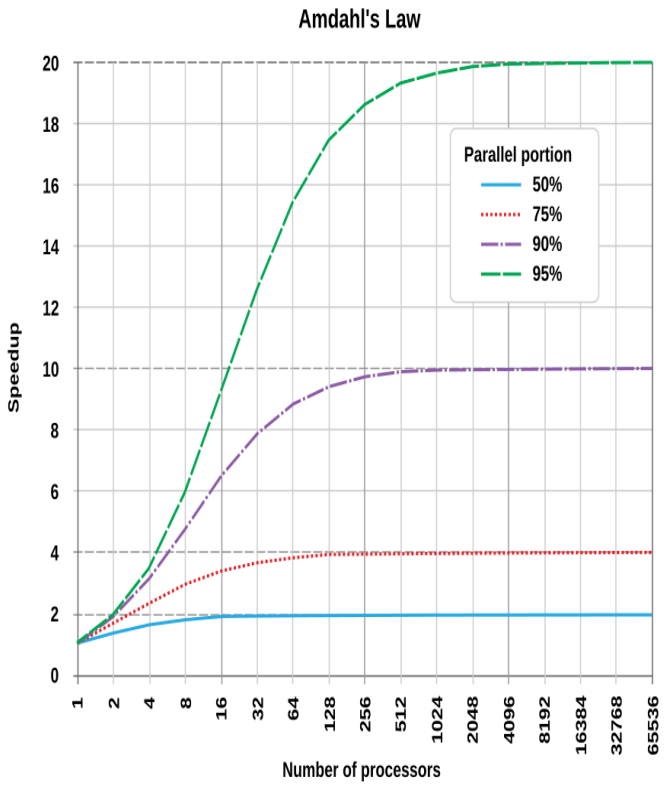




GIỚI THIỆU

Định luật Amdahl và Giới hạn Lý thuyết của Tăng tốc Song song

- **Nội dung định luật: Hệ số tăng tốc tối đa của một chương trình song song bị giới hạn nghiêm ngặt bởi tỷ lệ phần mã bắt buộc phải chạy tuần tự.**
- Công thức tính hệ số tăng tốc: $S(N) = 1 / [(1 - P) + (P / N)]$, trong đó P là tỷ lệ mã có thể song song hóa, N là số bộ vi xử lý.
- Giới hạn trần tiệm cận: Khi số lượng bộ vi xử lý N tăng tới vô cực, hệ số tăng tốc tối đa chỉ đạt mức 1 / (1 - P).
- Hệ quả kỹ thuật: Nếu chỉ 10% khối lượng công việc mang tính tuần tự, hệ số tăng tốc tối đa sẽ không bao giờ vượt quá 10 lần, dù trang bị 1.000 vi xử lý.



GIỚI THIỆU

Cấu trúc Phân cấp Bộ nhớ và Hiện tượng Chia sẻ Sai trên Hệ Đa nhân

• Nút thắt truy xuất bộ nhớ: Tốc độ truy xuất dữ liệu từ bộ nhớ chính là yếu tố chi phối trực tiếp đến hiệu năng thực tế của các ứng dụng tính toán song song.

• Hiện tượng chia sẻ sai: Xảy ra khi các nhân vi xử lý khác nhau cập nhật các biến độc lập nhưng vô tình nằm chung trên một dòng bộ đệm ẩn (Cache Line 64 byte), làm vô hiệu hóa bộ đệm liên tục.

• Hậu quả: Tăng quá nhiều đơn vị thực thi vượt ngưỡng dung nạp sẽ làm suy giảm hiệu năng do tắc nghẽn đường truyền bộ nhớ.

Cấp bậc Bộ nhớ	Độ trễ Chu kỳ Vi xử lý	Băng thông Dữ liệu
Bộ đệm Cấp 1 (L1 Cache)	3 - 4 chu kỳ (~1 ns)	~1.000 GB/giây (Siêu tốc độ)
Bộ đệm Cấp 2 (L2 Cache)	10 - 12 chu kỳ (~3 ns)	~500 GB/giây
Bộ đệm Dùng chung Cấp 3 (L3 Cache)	30 - 40 chu kỳ (~12 ns)	~200 GB/giây
Bộ nhớ Chính (Main DRAM)	150 - 250 chu kỳ (~70 ns)	~30 - 50 GB/giây (Nút thắt cổ chai)

GIỚI THIỆU

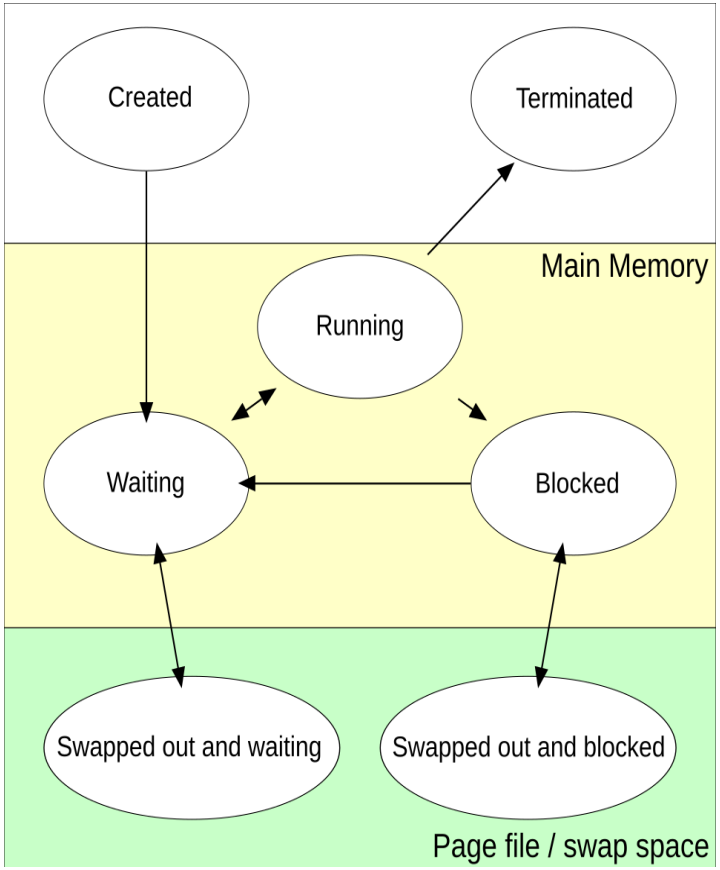
Mô hình hóa Trạng thái: Thuộc tính An toàn và Thuộc tính Hoạt tính

• Bản chất của mô hình đồng thời: Quản lý và kiểm soát sự tương tác phức tạp giữa các luồng thực thi độc lập diễn ra trong cùng một không gian trạng thái.

• Thuộc tính An toàn (Safety): Đảm bảo không có trạng thái sai lệch nào xảy ra trong suốt quá trình vận hành; bảo vệ hệ thống khỏi bế tắc tài nguyên (Deadlock) hay sai lệch dữ liệu.

• Thuộc tính Hoạt tính (Liveness): Đảm bảo các trạng thái mong muốn cuối cùng sẽ diễn ra; ngăn chặn hiện tượng một tiến trình bị bỏ đói tài nguyên vĩnh viễn (Starvation).

• Giá trị học thuật: Thiết lập cơ sở lý thuyết toán học vững chắc để thẩm định và chứng minh tính đúng đắn của các giải thuật đồng thời và phân tán.



GIỚI THIỆU

Bảng Đối chiếu Đặc tính Kỹ thuật: Tiến trình • Luồng • Hàm đồng quy

Tiêu chí Đối chiếu	Tiến trình (Process)	Luồng (Thread)	Hàm đồng quy (Coroutine)
Tầng quản lý	Nhân hệ điều hành (Kernel)	Nhân hệ điều hành (Kernel)	Không gian người dùng (User-space)
Cơ chế Lập lịch	Tranh đoạt ưu tiên (Preemptive)	Tranh đoạt ưu tiên (Preemptive)	Hợp tác tự nguyện (Cooperative)
Không gian bộ nhớ	Cô lập hoàn toàn (Private)	Chia sẻ chung Vùng nhớ tĩnh/Động	Chia sẻ chung bộ nhớ Luồng sở tại
Bộ nhớ RAM	~15–30 MB / tiến trình	~8 MB ngăn xếp ảo, ~100KB thực	~1–2 KB / hàm đồng quy siêu nhẹ
Thời gian tạo mới	~10–50 mili-giây (Rất nặng)	~50–200 micro-giây	< 1 micro-giây (Siêu tốc độ)
Chuyển ngữ cảnh	~5–15 micro-giây (Xóa TLB)	~1–3 micro-giây (Lưu thanh ghi)	~20–50 nano-giây (Đổi con trỏ)
Khai thác Đa lõi	Tuyệt đối (Mỗi tiến trình 1 core)	Bị hạn chế bởi khóa GIL CPython	Chạy trên 1 core (Vòng lặp đơn)
Kênh giao tiếp	Truyền thông liên tiến trình (IPC)	Truy xuất bộ nhớ chung trực tiếp	Biến trực tiếp / Hàng đợi bất đồng bộ
Rủi ro tiềm ẩn	Bế tắc giao tiếp, cạn kiệt RAM	Tranh chấp dữ liệu, Bế tắc khóa	Đóng băng vòng lặp nếu gọi hàm chặn
Miền tối ưu nhất	Tính toán nặng CPU thuần Python	Chờ Vào/Ra khi thư viện chặn	Vào/Ra quy mô lớn (Hàng vạn kết nối)

GIỚI THIỆU

Ma trận Phân loại Kiến trúc Vào/Ra: Đồng bộ vs Bất đồng bộ

Mô hình Kết hợp	Cơ chế Hoạt động	Tác động Hiệu năng	Minh họa Cú pháp Python

1. Đồng bộ + Phong tỏa (Sync Blocking)	Bên gọi chờ kết quả; luồng bị hệ điều hành đưa vào trạng thái ngủ.	Đơn giản nhưng lãng phí nghiêm trọng chu kỳ vi xử lý khi có trễ Vào/Ra.	`import requests`
			`resp = requests.get(url)`
			`# Chặn cứng luồng thực thi`

2. Đồng bộ + Phi phong tỏa (Sync Non-blocking)	Bên gọi nhận tín hiệu ngay; lặp lại việc thăm dò trạng thái liên tục.	Tạo tải ảo, làm vi xử lý tăng 100% chỉ để thăm dò kết quả sẵn sàng.	`sock.setblocking(False)`
			`while True:`
			`  try: data=recv(); break`
			`  except: pass`

3. Bất đồng bộ + Phong tỏa (Phản mẫu Thiết kế)	Khai báo hàm async nhưng bên trong gọi lời gọi hàm phong tỏa.	LỖI CHÍ MẠNG: Đóng băng toàn bộ Vòng lặp Sự kiện, tê liệt hệ thống!	`async def get():`
			`  time.sleep(2) # ĐÓNG BĂNG!`
			`  return 'ok'`

4. Bất đồng bộ + Phi phong tỏa (Kiến trúc Chuẩn mực)	Bên gọi nhận đối tượng tương lai ngay; luồng tự do phục vụ tiếp.	Hiệu năng tối đa: 1 luồng phục vụ hàng vạn kết nối đồng thời mượt mà.	`async def get():`
			`  await asyncio.sleep(2)`
			`  return 'ok'`

GIỚI THIỆU

Bốn Ngộ nhận Phổ biến trong Lập trình Đồng thời và Bất đồng bộ

Ngộ nhận Phổ biến	Sự thật Kỹ thuật	Bản chất Cơ chế
Ngộ nhận 1: Chuyển sang bất đồng bộ chắc chắn giúp chương trình tính toán nhanh hơn.	SAI. Bất đồng bộ không làm một phép tính nhanh hơn; chỉ giải phóng thời gian chờ Vào/Ra.	Với tác vụ nặng tính toán (CPU-bound), chi phí điều phối của vòng lặp sự kiện làm chậm hơn tuần tự.
Ngộ nhận 2: Càng tăng nhiều đơn vị thực thi trong nhóm luồng thì tốc độ càng tăng.	SAI. Vượt quá số nhân vật lý của vi xử lý sẽ gây suy thoái hiệu năng nghiêm trọng.	Hiện tượng cấp phát vượt ngưỡng: Hệ điều hành mất phần lớn thời gian cho việc hoán đổi ngữ cảnh.
Ngộ nhận 3: Bất đồng bộ chạy trên một luồng nên hoàn toàn không xảy ra tranh chấp dữ liệu.	SAI HOÀN TOÀN. Hiện tượng tranh chấp dữ liệu vẫn xảy ra ở tầng ứng dụng tại các điểm await.	Hai hàm đồng quy cùng đọc trạng thái, cùng nhường quyền tại await và cùng ghi đè làm sai lệch dữ liệu.
Ngộ nhận 4: Bất đồng bộ trên một máy tính tương đương với một hệ thống phân tán.	SAI VỀ BẢN CHẤT KIẾN TRÚC. Đây là hai mô hình tính toán hoàn toàn khác biệt.	Trên một máy tính luôn dùng chung một không gian bộ nhớ vật lý, không có độ trễ phân mảnh đường truyền mạng.

GIỚI THIỆU

Tổng kết Khung Kiến trúc Tính toán & Cầu nối Thực hành Python

• Khái quát Khung Lý thuyết & Kim chỉ nam Thiết kế Hệ thống:

• Phân định rõ ràng bốn mô hình: Tuần tự (đơn tuyến tính), Đồng thời (phân chia lát cắt thời gian), Song song (đa nhân vật lý thực thụ) và Bất đồng bộ (hướng sự kiện phi phong tỏa).

• Xác định đúng trọng tâm bài toán kỹ thuật: Tác vụ nghẽn vi xử lý (CPU-bound) cần giải phóng qua Đa tiến trình (Multiprocessing); Tác vụ nghẽn đường truyền mạng hoặc ổ đĩa (I/O-bound) cần tối ưu qua Hàm đồng quy và Vòng lặp sự kiện (Asyncio).

• Phân biệt tầng kiến trúc: Ranh giới bản chất giữa Lập lịch ưu tiên của Nhân hệ điều hành (Kernel Preemption) và Lập lịch hợp tác ở Không gian người dùng (User-space Cooperative Scheduling).

• Cầu nối sang kỹ thuật thực thi: Nền tảng lý thuyết này mở đường trực tiếp sang triển khai thực tế trên Python thông qua module concurrent.futures (ThreadPoolExecutor, ProcessPoolExecutor) và thư viện chuẩn asyncio.